

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности
Направление: 02.03.01 Математика и компьютерные науки

Сети ЭВМ и телекоммуникации компьютерных сетей

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3

«Организация межпроцессорного взаимодействия с помощью очереди
сообщений RabbitMQ. Retry с экспоненциальной задержкой»

Студент,
группы 5130201/30102

_____ Михайлова А. А.

Преподаватель

_____ Мулюха В. А.

«_____» _____ 2026 г.

Санкт-Петербург, 2026

1 Постановка задачи

Используя docker создать контейнеры, необходимые для реализации следующего функционала с использованием RabbitMQ/Kafka, а также показать, как именно осуществляется передача в этих условиях:

33. Retry с экспоненциальной задержкой. Потребитель, получивший сообщение, может отклонить его с требованием повторной доставки через увеличивающиеся интервалы (например, 1, 2, 4, 8 секунд). Реализовать конвейер: сообщение после неудачной обработки отправляется в очередь повторных попыток с TTL, а затем — в основную очередь.

2 Описание RabbitMQ

Брокер сообщений — программное обеспечение, которое позволяет выстроить действия в информационных системах таким образом, чтобы обеспечить асинхронный обмен сообщениями между сервисами.

Асинхронный обмен предполагает отправку запроса или сообщения от одного сервиса к другому, при этом деятельность сервиса-отправителя не приостанавливается в ожидании ответа от получателя.

Для обеспечения асинхронной доставки сообщений используется специальное программное обеспечение — брокер сообщений.

RabbitMQ - брокер сообщений с открытым исходным кодом, который реализует протокол AMQP.

RabbitMQ состоит из нескольких основных компонентов:

- **Queue** — структура данных, где хранятся сообщения до того, как получатель их обработает.
- **Message** — единица данных, которая передается от отправителя (Producer) к получателю (Consumer) через очередь.
- **Exchange** — обменник. Компонент, который осуществляет маршрутизацию, т.е. распределение данных по очередям на основе binding.
- **Binding** — связь между обменником и очередью, которая определяет, какие сообщения в какую очередь будут направлены.
- **Producer** — сервис, отправляющий данные.
- **Consumer** — сервис, который получает и обрабатывает данные.

3 Предварительная настройка

3.1 Установка Docker

1. Перейдите на <https://www.docker.com/products/docker-desktop>;
2. Скачайте Docker Desktop для вашей ОС;
3. Установите и запустите Docker Desktop.

3.2 Настройка Docker Compose

Docker Compose – это конфигурационный файл, в котором описываются все контейнеры, нужные для запуска приложения с помощью Docker Compose. Он позволяет одной командой запускать несколько контейнеров, указывая, как они должны работать вместе.

```
1 services:
2
3   rabbitmq:
4     image: rabbitmq:3-management
5     ports:
6       - "5672:5672"
7       - "15672:15672"
8     environment:
9       RABBITMQ_DEFAULT_USER: user
10      RABBITMQ_DEFAULT_PASS: password
11     healthcheck:
12       test: ["CMD", "rabbitmq-diagnostics", "ping"]
13       interval: 10s
14       timeout: 5s
15       retries: 5
16
17   producer:
18     build: ./producer
19     depends_on:
20       rabbitmq:
21         condition: service_healthy
22     environment:
23       - MESSAGES_COUNT=10
24       - SEND_INTERVAL=2
25       - PYTHONUNBUFFERED=1
26
27   consumer:
28     build: ./consumer
29     depends_on:
30       rabbitmq:
31         condition: service_healthy
32     environment:
33       - MAX_RETRIES=4
```

3.3 producer

3.3.1 Настройка Dockerfile

Инструкция для создания Docker-образа.

```

1 FROM python:3.10-slim
2
3 WORKDIR /app
4
5 COPY producer.py .
6 RUN pip install pika
7
8 CMD ["python", "producer.py"]

```

3.3.2 Реализация producer

```

1 import pika
2 import os
3 import time
4 import json
5 from datetime import datetime
6
7 MESSAGES_COUNT = int(os.environ.get("MESSAGES_COUNT", "10"))
8 SEND_INTERVAL = int(os.environ.get("SEND_INTERVAL", "2"))
9
10 def connect_to_rabbitmq(retries=10, delay=5):
11     credentials = pika.PlainCredentials('user', 'password')
12     parameters = pika.ConnectionParameters('rabbitmq',
13     credentials=credentials)
14     for attempt in range(retries):
15         try:
16             return pika.BlockingConnection(parameters)
17         except pika.exceptions.AMQPConnectionError:
18             print(f"[Producer] RabbitMQ not ready, retrying in {
19             delay}s... "
20             f"(attempt {attempt + 1}/{retries})")
21             time.sleep(delay)
22     raise Exception("Could not connect to RabbitMQ")
23
24 def setup_topology(channel):
25     channel.exchange_declare(
26         exchange='main_exchange',
27         exchange_type='direct',
28         durable=True
29     )
30
31     channel.queue_declare(queue='main_queue', durable=True)

```

```

30     channel.queue_bind(
31         queue='main_queue',
32         exchange='main_exchange',
33         routing_key='main'
34     )
35
36     channel.queue_declare(queue='dead_letter_queue', durable=True
37         )
38
39     delays = [1, 2, 4, 8]
40     for sec in delays:
41         channel.queue_declare(
42             queue=f'retry_{sec}s',
43             durable=True,
44             arguments={
45                 'x-message-ttl': sec * 1000,
46                 'x-dead-letter-exchange': 'main_exchange',
47                 'x-dead-letter-routing-key': 'main'
48             }
49         )
50
51     print("[Producer] Topology declared successfully")
52
53     connection = connect_to_rabbitmq()
54     channel = connection.channel()
55     setup_topology(channel)
56
57     print(f"[Producer] Will send {MESSAGES_COUNT} messages "
58         f"every {SEND_INTERVAL}s\n")
59
60     for i in range(1, MESSAGES_COUNT + 1):
61         body = json.dumps({
62             "id": i,
63             "payload": f"Task #{i}",
64             "created_at": datetime.now().strftime('%Y-%m-%d %H:%M:%S')
65         })
66
67         properties = pika.BasicProperties(
68             delivery_mode=2,
69             headers={"retry_count": 0}
70         )
71
72         channel.basic_publish(
73             exchange='main_exchange',
74             routing_key='main',
75             body=body,
76             properties=properties
77         )
78
79     ts = datetime.now().strftime('%H:%M:%S')
80     print(f"[Producer] [{ts}] Sent message #{i}: Task #{i}")

```

```

80     time.sleep(SEND_INTERVAL)
81
82 print("[Producer] All messages sent. Exiting.")
83 connection.close()

```

3.4 consumer

3.4.1 Настройка Dockerfile

```

1 FROM python:3.10-slim
2
3 WORKDIR /app
4
5 COPY consumer.py .
6 RUN pip install pika
7
8 CMD ["python", "-u", "consumer.py"]

```

3.4.2 Реализация consumer

```

1 import pika
2 import os
3 import time
4 import json
5 import random
6 from datetime import datetime
7
8 MAX_RETRIES      = int(os.environ.get("MAX_RETRIES", "4"))
9 FAIL_PROBABILITY = float(os.environ.get("FAIL_PROBABILITY", "0.7"
10 ))
11
12 RETRY_DELAYS = [1, 2, 4, 8]
13
14 def connect_to_rabbitmq(retries=10, delay=5):
15     credentials = pika.PlainCredentials('user', 'password')
16     parameters = pika.ConnectionParameters('rabbitmq',
17     credentials=credentials)
18     for attempt in range(retries):
19         try:
20             return pika.BlockingConnection(parameters)
21         except pika.exceptions.AMQPConnectionError:
22             print(f"[Consumer] RabbitMQ not ready, retrying in {
23             delay}s... "
24             f"(attempt {attempt + 1}/{retries})")
25             time.sleep(delay)
26     raise Exception("Could not connect to RabbitMQ")
27
28 def setup_topology(channel):
29     channel.exchange_declare(

```

```

27         exchange='main_exchange',
28         exchange_type='direct',
29         durable=True
30     )
31     channel.queue_declare(queue='main_queue', durable=True)
32     channel.queue_bind(
33         queue='main_queue',
34         exchange='main_exchange',
35         routing_key='main'
36     )
37     channel.queue_declare(queue='dead_letter_queue', durable=True)
38
39     delays = [1, 2, 4, 8]
40     for sec in delays:
41         channel.queue_declare(
42             queue=f'retry_{sec}s',
43             durable=True,
44             arguments={
45                 'x-message-ttl': sec * 1000,
46                 'x-dead-letter-exchange': 'main_exchange',
47                 'x-dead-letter-routing-key': 'main'
48             }
49         )
50     print("[Consumer] Topology declared successfully")
51
52
53 def process_message(message: dict) -> bool:
54     """
55     Возвращает True при успехе, False при ошибке.
56     С вероятностью FAIL_PROBABILITY симулирует сбой.
57     """
58     time.sleep(0.5)
59     return random.random() >= FAIL_PROBABILITY
60
61
62 def on_message(ch, method, properties, body):
63     ts = datetime.now().strftime('%H:%M:%S')
64     message = json.loads(body)
65     headers = properties.headers or {}
66     retry_count = int(headers.get("retry_count", 0))
67
68     print(f"\n[Consumer] [{ts}] Received message #{message['id']}")
69
70     f"(retry_count={retry_count}): {message['payload']}"
71
72     success = process_message(message)
73
74     if success:
75         ch.basic_ack(delivery_tag=method.delivery_tag)
76         print(f"[Consumer] [{ts}] Message #{message['id']} processed successfully")

```

```

76
77     else:
78         ch.basic_ack(delivery_tag=method.delivery_tag)
79
80         if retry_count >= MAX_RETRIES:
81             print(f"[Consumer] [{ts}]      Message #{message['id']}
82                   ']} FAILED "
83                   f"after {retry_count} retries      sending to
84                   dead_letter_queue")
85
86             ch.basic_publish(
87                 exchange='',
88                 routing_key='dead_letter_queue',
89                 body=body,
90                 properties=pika.BasicProperties(
91                     delivery_mode=2,
92                     headers={**headers, "retry_count":
93                             retry_count, "reason": "
94                             max_retries_exceeded"}
95                 )
96             )
97
98         else:
99             delay_sec = RETRY_DELAYS[retry_count]
100             retry_queue = f'retry_{delay_sec}s'
101             new_retry_count = retry_count + 1
102
103             print(f"[Consumer] [{ts}]      Message #{message['id']}
104                   ']} FAILED "
105                   f"(attempt {retry_count + 1}/{MAX_RETRIES}) "
106                   f"      retry in {delay_sec}s via [{retry_queue}]
107                   ")
108
109             ch.basic_publish(
110                 exchange='',
111                 routing_key=retry_queue,
112                 body=body,
113                 properties=pika.BasicProperties(
114                     delivery_mode=2,
115                     headers={**headers, "retry_count":
116                             new_retry_count}
117                 )
118             )
119
120             print(f"[Consumer] [{ts}]      Message #{message['id']}
121                   will be re-queued "
122                   f"to main_queue after {delay_sec}s TTL expires"
123                   )
124
125 connection = connect_to_rabbitmq()
126 channel    = connection.channel()
127 setup_topology(channel)

```



```

119
120 channel.basic_qos(prefetch_count=1)
121
122 channel.basic_consume(
123     queue='main_queue',
124     on_message_callback=on_message,
125     auto_ack=False
126 )
127
128 print(f"[Consumer] Listening on main_queue "
129       f"(fail_prob={FAIL_PROBABILITY}, max_retries={MAX_RETRIES},
130       f"delays={RETRY_DELAYS[:MAX_RETRIES]}s)\n")
131
132 channel.start_consuming()

```

4 Запуск

1. Запустить Docker Desktop
2. Перейти в папку
C:\Users\user\OneDrive\Документы\бсем\сети\retry_rabbitmq в терминале
3. Выполнить команды
docker-compose down -v
docker-compose up --build
4. Во втором терминале
docker-compose logs -f consumer
5. Остановка
docker-compose down

5 Результаты работы

Логи при попадании в dead_letter_queue

```
producer-1 | [Producer] [20:08:50] Sent message #1: Task #1
```

```
consumer-1 | [Consumer] [20:08:50]
Received message #1 (retry_count=0): Task #1
```

```
consumer-1 | [Consumer] [20:08:50]
Message #1 FAILED (attempt 1/4) → retry in 1s via [retry_1s]
```

```
consumer-1 | [Consumer] [20:08:50]
Message #1 will be re-queued to main_queue after 1s TTL expires
```

```
consumer-1 |  
  
consumer-1 | [Consumer] [20:08:51]  
Received message #1 (retry_count=1): Task #1  
  
consumer-1 | [Consumer] [20:08:51]  
Message #1 FAILED (attempt 2/4) → retry in 2s via [retry_2s]  
  
consumer-1 | [Consumer] [20:08:51]  
Message #1 will be re-queued to main_queue after 2s TTL expires  
  
consumer-1 |  
  
consumer-1 | [Consumer] [20:08:54]  
Received message #1 (retry_count=2): Task #1  
  
consumer-1 | [Consumer] [20:08:54]  
Message #1 FAILED (attempt 3/4) → retry in 4s via [retry_4s]  
  
consumer-1 | [Consumer] [20:08:54]  
Message #1 will be re-queued to main_queue after 4s TTL expires  
  
consumer-1 |  
  
consumer-1 | [Consumer] [20:08:59]  
Received message #1 (retry_count=3): Task #1  
  
consumer-1 | [Consumer] [20:08:59]  
Message #1 FAILED (attempt 4/4) → retry in 8s via [retry_8s]  
  
consumer-1 | [Consumer] [20:08:59]  
Message #1 will be re-queued to main_queue after 8s TTL expires  
  
consumer-1 | [Consumer] [20:09:08]  
Received message #1 (retry_count=4): Task #1  
  
consumer-1 | [Consumer] [20:09:08]  
Message #1 FAILED after 4 retries → sending to dead_letter_queue  
  
Логги при попадании в main_queue  
  
producer-1 | [Producer] [20:09:04] Sent message #8: Task #8
```

```
consumer-1 | [Consumer] [20:09:04]  
Received message #8 (retry_count=0): Task #8  
consumer-1 | [Consumer] [20:09:04]  
Message #8 processed successfully
```

6 Результаты работы

В результате выполнения лабораторной работы реализован механизм повторной доставки сообщений с экспоненциальной задержкой с использованием RabbitMQ. Retry-очереди с TTL 1, 2, 4, 8 сек автоматически отправляют сообщения в основную очередь по истечении времени ожидания. Сообщения, которые превысили максимальное число попыток, направляются в `dead_letter_queue`.